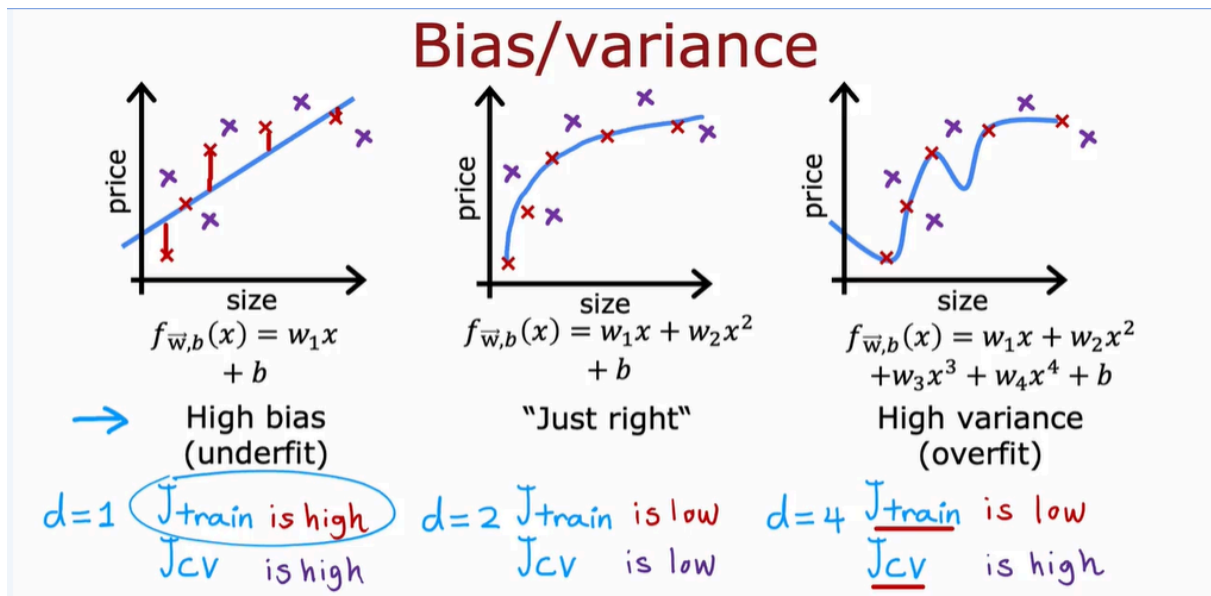


Bias and variance

Diagnosing bias and variance



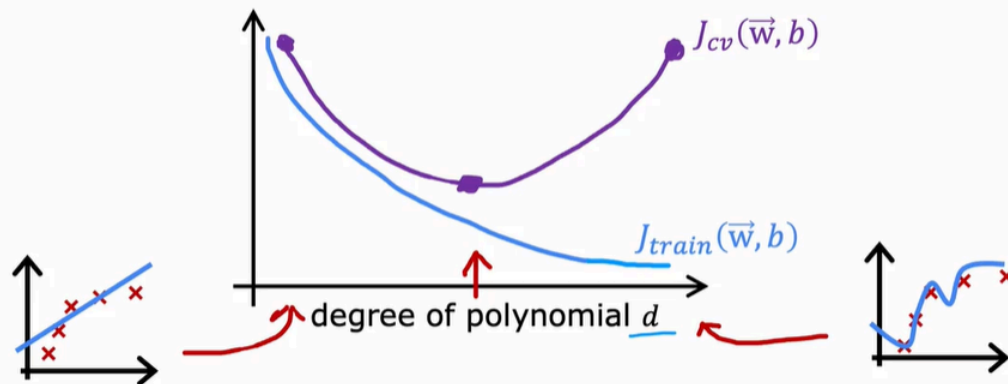
The model is underfit (high bias) when both:

- J_{train} is high
- J_{cv} is high

The model is overfit when:

- J_{cv} is much higher than J_{train}

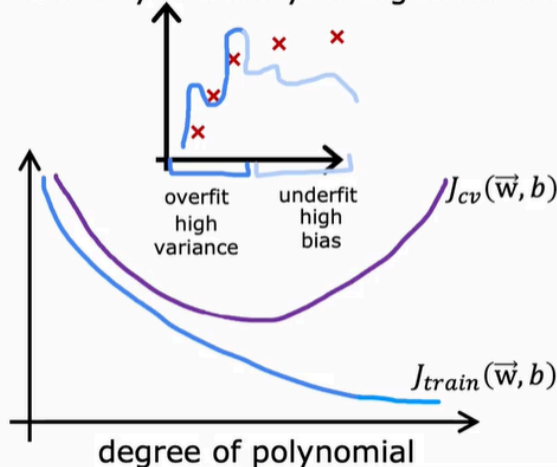
Understanding bias and variance



We should choose the degree of polynomial d at the red arrow.

Diagnosing bias and variance

How do you tell if your algorithm has a bias or variance problem?



High bias (underfit)

J_{train} will be high
($J_{train} \approx J_{cv}$)

High variance (overfit)

$J_{cv} \gg J_{train}$
(J_{train} may be low)

High bias and high variance

J_{train} will be high
and $J_{cv} \gg J_{train}$

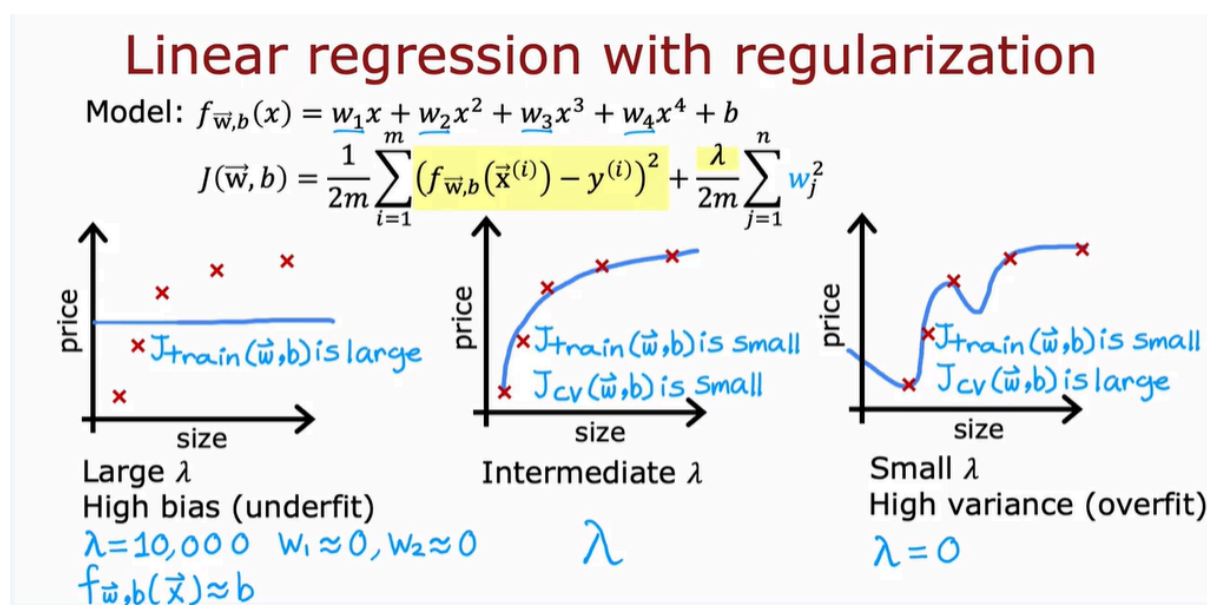
- High bias means is not even doing well on the training set
- High variance means, it does much worse on the cross validation set than the training set.

Whenever I'm training a machine learning algorithm, I will almost always try to figure out to understand algorithm has a high bias or underfitting versus a high-variance when overfitting problem. This will give good guidance, as we'll see later this week, on how you can improve the performance of the algorithm.

Regularization and bias/variance

Recall:

Regularization in machine learning is a technique used to **prevent overfitting** by adding a penalty to the loss function of a model. Overfitting happens when a model learns the training data too well, including its noise, and performs poorly on new, unseen data.



Note:

Lambda is the regularization parameter that controls how much you trade-off keeping the parameters w small versus fitting the training data well.

Reminder:

The problem we're addressing is if you're fitting a fourth-order polynomial, so that's the model and you're using regularization, how can you choose a good value of Lambda?

Choosing the regularization parameter λ

Model: $f_{\vec{w},b}(x) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$

- $\rightarrow$ 1. Try $\lambda = 0 \rightarrow \min_{\vec{w},b} J(\vec{w},b) \rightarrow w^{<1>}, b^{<1>} \rightarrow J_{cv}(w^{<1>}, b^{<1>})$
 $\rightarrow$ 2. Try $\lambda = 0.01 \rightarrow w^{<2>}, b^{<2>} \rightarrow J_{cv}(w^{<2>}, b^{<2>})$
 $\rightarrow$ 3. Try $\lambda = 0.02 \rightarrow J_{cv}(w^{<3>}, b^{<3>})$
 $\rightarrow$ 4. Try $\lambda = 0.04 \rightarrow J_{cv}(w^{<5>}, b^{<5>})$
 $\rightarrow$ 5. Try $\lambda = 0.08$
 $\vdots$
 $\rightarrow$ 12. Try $\lambda \approx 10 \rightarrow w^{<12>}, b^{<12>} \rightarrow J_{cv}(w^{<12>}, b^{<12>})$

Pick $w^{<5>}, b^{<5>}$

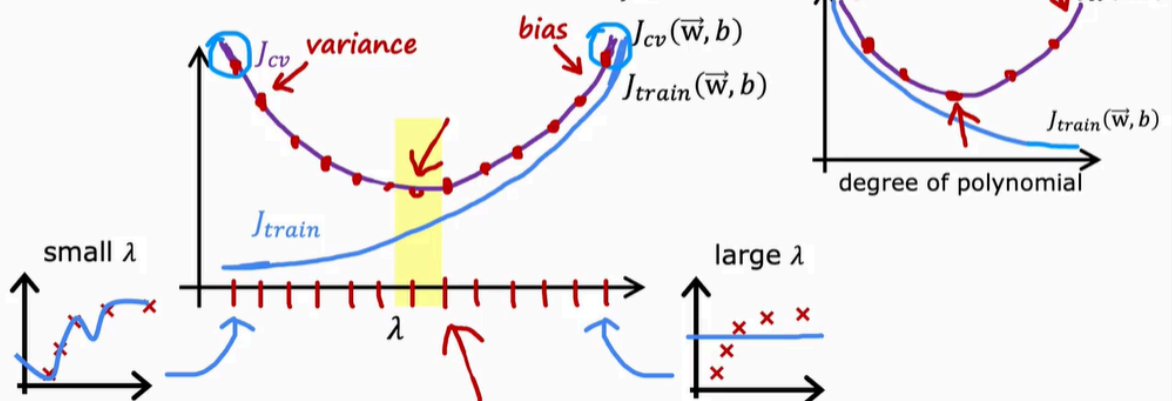
Report test error: $J_{test}(w^{<5>}, b^{<5>})$

We double Lambda in each step, then choose the most lowest J_{cv}

That could be the Lambda that we need.

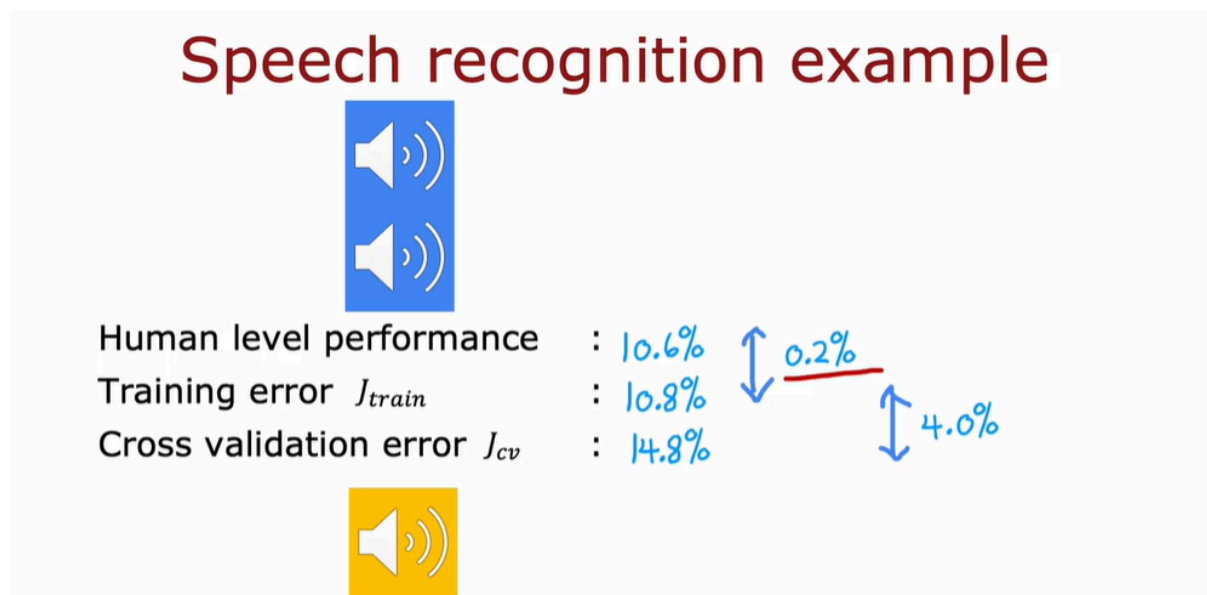
Bias and variance as a function of regularization parameter λ

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$



But in both cases (the choice of Lambda and the degree of polynomial), cross-validation, evaluating different values can help you choose a good value of d or a good value of Lambda. That's how the choice of regularization parameter Lambda affects the bias and variance and overall performance of your algorithm, and you've also seen how you can use cross-validation to make a good choice for the regularization parameter Lambda.

Establishing a baseline level of performance



A lot of users doing web search on a mobile phone will use speech recognition rather than type on the tiny keyboards on our phones because speaking to a phone is often faster than typing. Typical audio that's a web search engine we get would be like this, "What is today's weather?" Or like this, "Coffee shops near me."

It's the job of the speech recognition algorithms to output the transcripts whether it's today's weather or coffee shops near me. Now, if you were to train a speech recognition system and measure the training error, and the training error means what's the percentage of audio clips in your training set that the algorithm does not transcribe correctly in its entirety.

- Let's say the training error for this data-set is 10.8 percent meaning that it transcribes it perfectly for 89.2% of your training set, but makes some mistake in 10.8% of your training set.
- If you were to also measure your speech recognition algorithm's performance on a separate cross-validation set, let's say it gets 14.8% error. If you were to look at these numbers it looks like the training error is really high, it got 10% wrong, and then the cross-validation error is higher but getting 10% of even your training set wrong that seems pretty high. It seems like that 10 percent error would lead you to conclude it has high bias

because it's not doing well on your training set, but it turns out that when analyzing speech recognition it's useful to also measure one other thing which is what is the human level of performance?

In other words, how well can even humans transcribe speech accurately from these audio clips? Concretely, let's say that you measure how well fluent speakers can transcribe audio clips and you find that **human level performance achieves 10.6 percent error**. Why is human level error so high? It turns out that for web search, there are a lot of audio clips that sound like this, "I'm going to navigate to [inaudible]."

There's a lot of noisy audio where really no one can accurately transcribe what was said because of the noise in the audio. If even a human makes 10.6 percent error, then it seems difficult to expect a learning algorithm to do much better.

In order to judge if the training error is high, it turns out to be more useful to see if the training error is much higher than a human level of performance, and in this example it does just 0.2 percent worse than humans. Given that humans are actually really good at recognizing speech I think if I can build a speech recognition system that achieves 10.6 percent error matching human performance I'd be pretty happy, so it's just doing a little bit worse than humans. But in contrast, the gap or the difference between J_{cv} and J_{train} is much larger.

There's actually a four percent gap there, whereas previously we had said maybe 10.8 percent error means this is high bias. When we benchmark it to human level performance, we see that the algorithm is actually doing quite well on the training set, but the bigger problem is the cross-validation error is much higher than the training error which is why I would conclude that this algorithm actually has more of a variance problem than a bias problem.

Establishing a baseline level of performance

What is the level of error you can reasonably hope to get to?

- • Human level performance
- • Competing algorithms performance
- • Guess based on experience

One common way to establish a baseline level of performance is to measure how well humans can do on this task because humans are really good at understanding speech data, or processing images or understanding texts. Human level performance is often a good benchmark when you are using unstructured data, such as: audio, images, or texts.

Another way to estimate a baseline level of performance is if there's some competing algorithm, maybe a previous implementation that someone else has implemented or even a competitor's algorithm to establish a baseline level of performance if you can measure that, or sometimes you might guess based on prior experience. If you have access to this baseline level of performance that is, what is the level of error you can reasonably hope to get to or what is the desired level of performance that you want your algorithm to get to?

Bias/variance examples

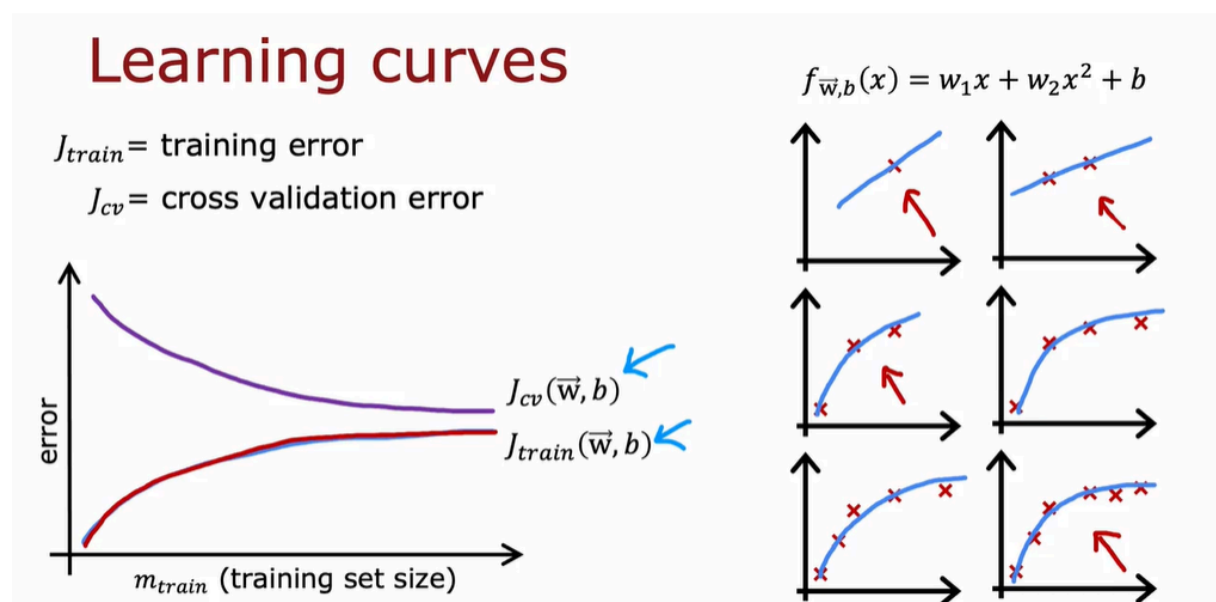
Baseline performance	: 10.6%	↑ 0.2%	10.6%	↑ 4.4%	10.6%	↑ 4.4%
Training error (J_{train})	: 10.8%	↓ 0.2%	15.0%	↓ 0.5%	15.0%	↓ 0.5%
Cross validation error (J_{cv})	: 14.8%	↓ 4.0%	15.5%	↓ 0.5%	19.7%	↓ 4.7%
			high variance	high bias	high bias	high variance

By looking at these numbers, training error and cross validation error, you can get a sense intuitively or informally of the degree to which your algorithm has a high bias or high variance problem. Just to summarize:

- This gap between these first two numbers (baseline performance error and training error) gives you a sense of whether you have a high bias problem, and
- The gap between these two numbers (Training error and Cross validation error) gives you a sense of whether you have a high variance problem. Sometimes the baseline level of performance could be zero percent.

If your goal is to achieve perfect performance than the baseline level of performance it could be zero percent, but for some applications like the speech recognition application where some audio is just noisy then the baseline level of a performance could be much higher than zero.

Learning curves

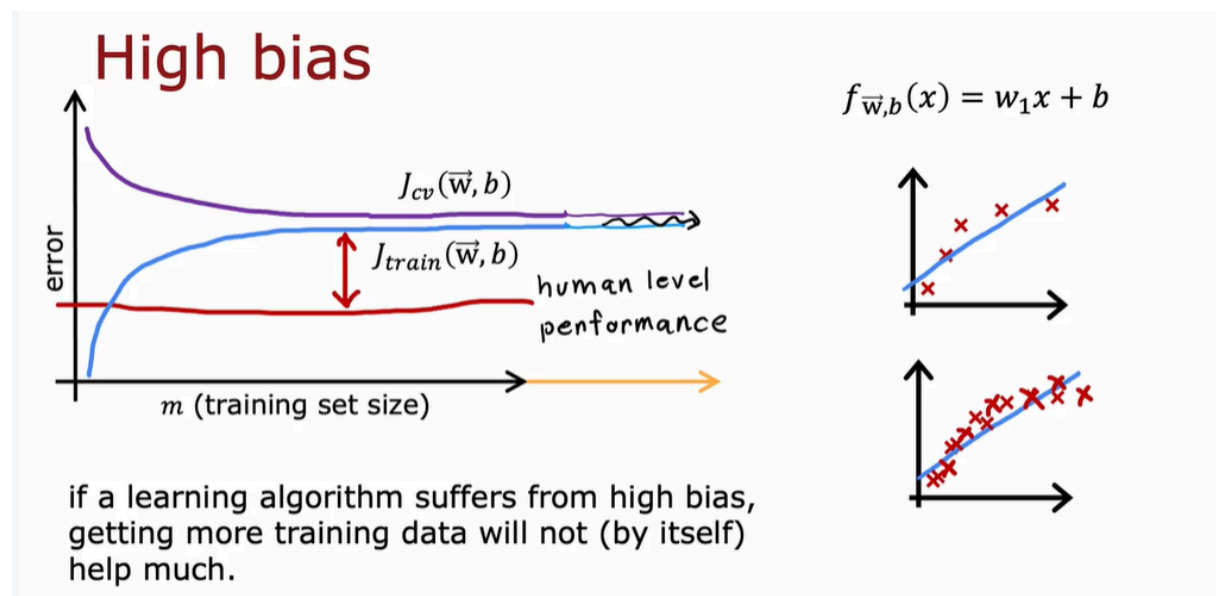


When you have a very small number of training examples like one or two or even three, is relatively easy to get zero or very small training error, but when you have a larger training set is harder for quadratic function to fit all the training examples perfectly. Which is why as the training set gets bigger, the training error increases because it's harder to fit all of the training examples perfectly.

Notice one other thing about these curves, which is the cross-validation error, will be typically higher than the training error because you fit the parameters to the training set. You expect to do at least a little bit better or when m is small, maybe even a lot better on the training set than on the trans validation set. Let's

now take a look at what the learning curves will look like for an algorithm with high bias versus one with high variance.

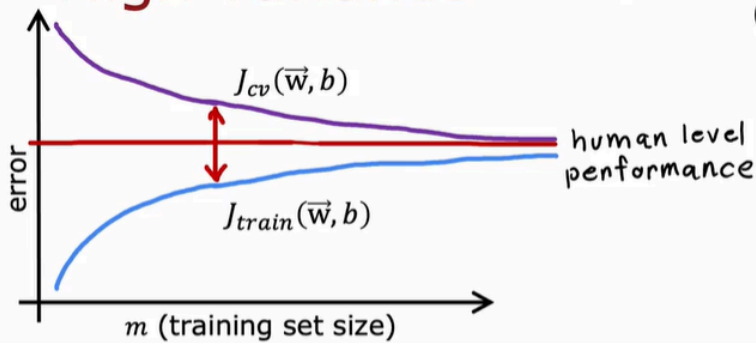
High bias example



In fact, this curve of training error may start to flatten out. We call it plateau, meaning flatten out after a while. That's because as you get more and more training examples when you're fitting the simple linear function, your model doesn't actually change that much more. It's fitting a straight line and even as you get more and more and more examples, there's just not that much more to change, which is why the average training error flattens out after a while.

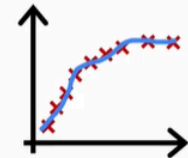
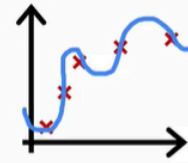
High variance example

High variance



$$f_{\vec{w},b}(x) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$$

(with small λ)



If a learning algorithm suffers from high variance, getting more training data is likely to help.

Deciding what to try next revisited

Debugging a learning algorithm

You've implemented regularized linear regression on housing prices

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

But it makes unacceptably large errors in predictions. What do you try next?

- Get more training examples
- Try smaller sets of features $x, x^2, x^3, x^4, x^5, \dots$
- Try getting additional features
- Try adding polynomial features $(x_1^2, x_2^2, x_1x_2, \text{etc})$
- Try decreasing λ
- Try increasing λ

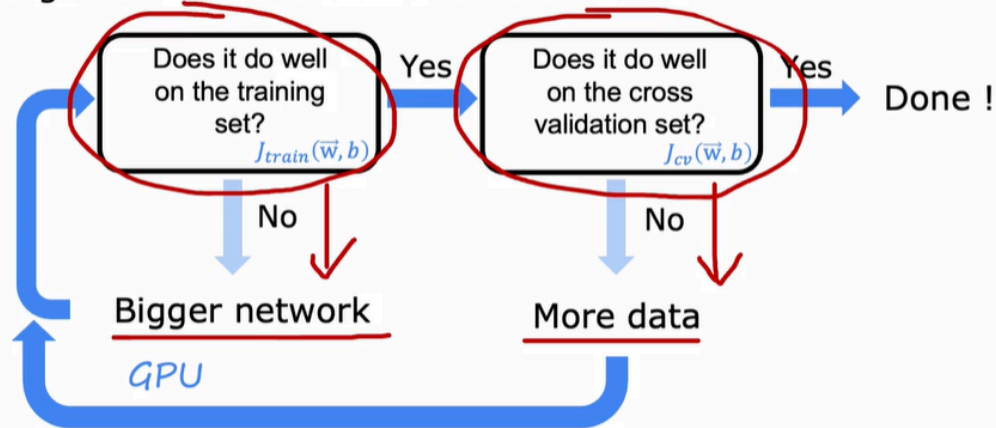
fixes high variance
 fixes high variance
 fixes high bias
 fixes high bias
 fixes high bias
 fixes high variance

Bias/variance and neural networks

Neural networks and bias variance

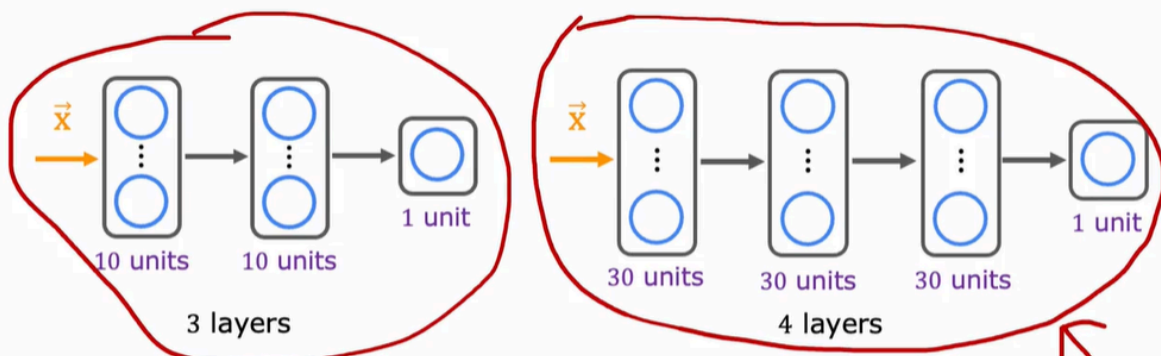
Neural networks and bias variance

Large neural networks are low bias machines



Neural networks and regularization

Neural networks and regularization



A large neural network will usually do as well or better than a smaller one so long as regularization is chosen appropriately.

Neural network regularization

Neural network regularization

$$J(\mathbf{W}, \mathbf{B}) = \underbrace{\frac{1}{m} \sum_{i=1}^m L(f(\vec{x}^{(i)}), y^{(i)})}_{\text{loss}} + \underbrace{\frac{\lambda}{2m} \sum_{\text{all weights } \mathbf{W}} (w^2)}_{\text{regularization}} \quad b$$

Unregularized MNIST model

```
layer_1 = Dense(units=25, activation="relu")
layer_2 = Dense(units=15, activation="relu")
layer_3 = Dense(units=1, activation="sigmoid")
model = Sequential([layer_1, layer_2, layer_3])
```

Regularized MNIST model

```
layer_1 = Dense(units=25, activation="relu", kernel_regularizer=L2(0.01))
layer_2 = Dense(units=15, activation="relu", kernel_regularizer=L2(0.01))
layer_3 = Dense(units=1, activation="sigmoid", kernel_regularizer=L2(0.01))
model = Sequential([layer_1, layer_2, layer_3])
```

So to summarize two Takeaways, I hope you have from this video are one. It hardly ever heard to have a larger neural network so long as you regularize appropriately.

one caveat being that having a larger neural network can slow down your algorithm. So maybe that's the one way it hurts, but it shouldn't hurt your algorithm's performance for the most part and in fact it could even help it significantly.

And second so long as your training set isn't too large. Then a neural network, especially large neural network is often a low bias machine. It just fits very complicated functions very well, which is why when I'm training neural networks, I find that I'm often fighting variance problems rather than bias problems, at least if the neural network is large enough. So the rise of deep learning has really changed the way that machine learning practitioners think about bias and variance. Having said that even when you're training a neural network measuring bias and variance and using that to guide what you do next is often a very helpful thing to do.